Lesson: Formatted I/O

CSE 4107 : Structured Programming I

Shahriar Ivan

The printf Function

- The printf function must be supplied with a ***format string,*** followed by any values that are to be inserted into the string during printing:

  printf(*string*, *expr*1, *expr*2, ...);
- The format string may contain both ordinary characters and ***conversion specifications,*** which begin with the % character
- A conversion specification (***Format specifier***) is a placeholder representing a value to be filled in during printing.
  - %d is used for int values
  - %f is used for float values

The printf Function

Use of printf:

```
int i, j;
float x, y;

i = 10;
j = 20;
x = 43.2892f;
y = 5527.0f;
printf("i = %d, j = %d, x = %f, y = %f\n", i, j, x, y);
```

- Ordinary characters in a format string are printed as they appear in the string; conversion specifications are replaced

Output: i = 10, j = 20, x = 43.289200, y = 5527.000000

The printf Function

- Compilers aren’t required to check that the number of conversion specifications in a format string matches the number of output items
- Too many conversion specifications:

```
printf("%d %d\n", i);   /*** WRONG ***/
```

- Too few conversion specifications:

```
printf("%d\n", i, j);   /*** WRONG ***/
```

The printf Function

- Compilers aren’t required to check that a conversion specification is appropriate
- If the programmer uses an incorrect specification, the program will produce meaningless output:

```
printf("%f %d\n", i, x);  /*** WRONG ***/
```

The printf Function

Format Specifier

- A conversion specification can have the form **%*m*.*pX*** or **%-*m*.*pX***, where ***m*** and ***p*** are integer constants and ***X*** is a letter
- Both ***m*** and ***p*** are optional; if ***p*** is omitted, the period that separates ***m*** and ***p*** is also dropped
- In the conversion specification **%10.2f**, ***m*** is **10**, ***p*** is **2**, and ***X*** is **f**
- In the specification **%10f**, ***m*** is **10** and ***p*** (along with the period) is missing, but in the specification **%.2f**, ***p*** is **2** and ***m*** is missing

The printf Function

Format Specifier

- The ***minimum field width, m***, specifies the minimum number of characters to print
- If the value to be printed requires fewer than ***m*** characters, it is right-justified within the field.
  - %4d displays the number 123 as •123. (• represents the space character.)
- If the value to be printed requires more than ***m*** characters, the field width automatically expands to the necessary size

The printf Function

Format Specifier

- Putting a minus sign in front of *m* causes left justification
  - The specification %-4d would display 123 as 123•

- The meaning of the ***precision, p***, depends on the choice of *X*, the ***conversion specifier***

The printf Function

Format Specifier

- The **d** specifier is used to display an integer in decimal form.
  - ***p*** indicates the minimum number of digits to display (extra zeros are added to the beginning of the number if necessary).
  - If ***p*** is omitted, it is assumed to be 1

The printf Function

Format Specifier

- Conversion specifiers for floating-point numbers:

  e — *Exponential format*. ***p*** indicates how many digits should appear after the decimal point (the default is 6). If ***p*** is **0**, no decimal point is displayed.

  f — "*Fixed decimal*" format. ***p*** has the same meaning as for the **e** specifier.

  d — Minimum number (***p***) of digits to display (append extra zeros at the beginning if needed)

The printf Function

Use of various Format Specifiers

- Show the use of format specifiers for both int and float
- Use %d, %5d, %-5d, %5.3d for 40
- Use %10.3f, %10.3e for 839.21

The printf Function

Escape Sequences

- The \n code that used in format strings is called an ***escape sequence***
- Control characters:
  - Alert (Bell) : \a
  - New line : \n
  - Tab : \t
- Special characters:
  - Backslash (\) : \\
  - Double Quotes (“) : \”
  - Example: printf(“Item\tUnit\tPurchase\n\tPrice\tDate\n”);

The scanf Function

- scanf reads input according to a particular format
- A scanf format string may contain both ordinary characters and conversion specifications
- Only format specifiers in most cases

```
int i,j;
float x,y;
scanf("%d %d %f %f", &i, &j, &x, &y);
```

- Sample output: `1  -20  .3  -4.0e3`

The scanf Function

- When using scanf, the programmer must check that the number of conversion specifications matches the number of input variables and that each conversion is appropriate for the corresponding variable
- Another trap involves the & symbol, which normally precedes each variable in a scanf call
- The & is usually (but not always) required, and it’s the programmer’s responsibility to remember to use it

The scanf Function

How scanf works

- scanf tries to match groups of input characters with conversion specifications in the format string
- For each conversion specification, scanf tries to locate an item of the appropriate type in the input data, skipping blank space if necessary
- scanf then reads the item, stopping when it reaches a character that can't belong to the item.
  - If the item was read successfully, scanf continues processing the rest of the format string.
  - If not, scanf returns immediately

The scanf Function

How scanf works

- As it searches for a number, scanf ignores ***white-space characters*** (space, horizontal and vertical tab, form-feed, and new-line)
- Example: `scanf("%d%d%f%f", &i, &j, &x, &y);`
- The numbers can be on one line or spread over several lines:

```
  1
-20  .3
 -4.0e3
```

The scanf Function

How scanf works

- scanf “peeks” at the final new-line without reading it
- To read an int
  - Search for digit / a plus sign / a minus sign
  - Read until non digit is found
- To read a float
  - A plus sign/minus sign (Optional), followed by
  - digits (possibly containing decimal point), followed by
  - an exponent e/E (optional), followed by
  - a plus/minus sign (Optional) and one or more digits

The scanf Function

Ordinary Characters in Format Strings

- When `scanf` encounters a character that can't be part of the current item, the character is "put back" to be read again during the scanning of the next input item or during the next call of `scanf`
- When it encounters one or more white-space characters in a format string, `scanf` reads white-space characters from the input until it reaches a non-white-space character (which is "put back")

The scanf Function

Ordinary Characters in Format Strings

- When it encounters a non-white-space character in a format string, scanf compares it with the next input character
  - If they match, scanf discards the input character and continues processing the format string
  - If they don't match, scanf puts the offending character back into the input, then aborts
- Examples:
  - scanf("%d/%d", &x, &y) // input: •5/•96 —> x=5, y=96
  - If the input is •5•/•96 , scanf fails, because the / in the format string doesn't match the space in the input

The scanf Function

Confusing printf with scanf

- Don't use ampersand in printf
  - printf("%d %d\n", &i,&j); // Wrong
- Don't use ordinary characters unless it is necessary
  - scanf("%d, %d", &i, &j); // Not error, but be careful
- Putting a new-line character at the end of a scanf format string is usually a bad idea
- If the format string is "%d\n", scanf will skip white space, read an integer, then skip to next *non-white-space* character -> cause interactive programs to ***hang***

The scanf Function

Confusing printf with scanf

- Consider a C program that prompts the user to enter two fractions and then displays their sum
- Fractions are customarily denoted as: *numerator/denominator*

Sample program output:

```
Enter first fraction: 5/6
Enter second fraction: 3/4
The sum is 38/24
```